



1. Problem Title & Link

Title: LeetCode 739 : Daily Temperatures

Link: [LeetCode 739 - Daily Temperatures](#)

2. Problem Statement (Short Summary)

You are given an integer array:

temperatures[]

For each day, return how many days you have to wait until a warmer temperature.

If no warmer day exists:

return 0

for that position.

In simple terms:

For every temperature, find the distance to the next greater temperature.

3. Examples (Input → Output)

Example 1

Input:

temperatures=

[73,74,75,71,69,72,76,73]

Output:

[1,1,4,2,1,1,0,0]

Explanation:

73 → next warmer:74 →1 day

74 → next warmer:75 →1 day

75 → next warmer:76 →4 days

Example 2

Input:

temperatures=[30,40,50,60]

Output:

[1,1,1,0]

Example 3

Input:

temperatures=[30,60,90]



Output:

[1,1,0]

4. Constraints

$1 \leq \text{temperatures.length} \leq 10^5$

$30 \leq \text{temperatures}[i] \leq 100$

Special restrictions:

- Need next warmer day
- Better than $O(n^2)$

5. Core Concept (Pattern / Topic)

Monotonic Stack

6. Thought Process (Step-by-Step Explanation)

Brute Force

For every day:

1. Search future temperatures
2. Find first warmer day
3. Compute distance

Time:

$O(n^2)$

Too slow.

Optimized Thinking

Observation:

We need:

Next Greater Element

Use a stack storing indices.

Maintain stack as:

Decreasing temperatures

Algorithm:

1. Traverse temperatures
2. While current temperature is greater than stack top:
 - Pop index
 - Calculate waiting days
3. Push current index



7. Visual / Intuition Diagram (ASCII Diagram)

Input:

[73,74,75,71,69,72,76,73]

Stack process:

73

74 arrives

74>73

Pop 73

answer[73]=1

75 arrives

75>74

Pop 74

answer[74]=1

72 arrives

72>69

Pop 69

answer=1

8. Pseudocode (Language Independent)

stack=[]

answer=[0]*n



```
for i in range(n):

    while stack not empty
    and
    current >
    temperatures[top]:

        index=pop()

        answer[index]=
        i-index

    push(i)

return answer
```

9. Code Implementation

Python

class Solution:

```
def dailyTemperatures(
    self,
    temperatures):

    stack=[]

    result=[0] * len(
        temperatures
    )

    for i in range(
        len(
            temperatures
        )
    ):

        while(
```



```
stack
and
temperatures[i]
>
temperatures[
    stack[-1]
]
):
```

```
index=
stack.pop()
```

```
result[index]=(
    i-index
)
```

```
stack.append(i)
```

```
return result
```

Java

```
class Solution {
```

```
    public int[] dailyTemperatures(
        int[] temperatures) {
```

```
        Stack<Integer>
            stack=
            new Stack<>();
```

```
        int[] result=
            new int[
                temperatures.length
            ];
```

```
        for(int i=0;
            i<
```



```
    temperatures.length;  
    i++){  
  
    while(  
        !stack.isEmpty()  
        &&  
        temperatures[i]  
        >  
        temperatures[  
            stack.peek()  
        ]  
    ){  
  
        int index=  
            stack.pop();  
  
        result[index]=  
            i-index;  
    }  
  
    stack.push(i);  
}  
  
return result;  
}
```

10. Time & Space Complexity

Time Complexity:

$O(n)$

Reason:

Each index is pushed and popped once.

Space Complexity:

$O(n)$

Reason:

Stack may store all indices.



11. Common Mistakes / Edge Cases

1. Storing temperatures instead of indices

Wrong:

stack=[73,74]

Need:

stack=[0,1]

because distance calculation requires indices.

2. Using:

>=

instead of:

>

Need strictly warmer day.

3. Forgetting default:

0

for no future warmer day.

4. Missing decreasing sequence

[80,70,60]

Output:

[0,0,0]

12. Detailed Dry Run (Step-by-Step Table)

Input:

[73,74,75,71,69,72,76,73]

Index	Temp	Stack	Action	Result
0	73	[0]	Push	[0,0,0,0,0,0,0,0]
1	74	[0]	Pop 0	[1,0,0,0,0,0,0,0]
2	75	[1]	Pop 1	[1,1,0,0,0,0,0,0]
5	72	[2,3,4]	Pop 4,3	[1,1,0,2,1,0,0,0]
6	76	[2,5]	Pop 5,2	[1,1,4,2,1,1,0,0]

Final:



[1,1,4,2,1,1,0,0]

13. Common Use Cases (Real-Life / Interview)

Weather forecasting systems

Stock price trend analysis

Signal processing

Time-series analysis

14. Common Traps (Important!)

1. Using nested loops
2. Storing values instead of indices
3. Using incorrect comparison operator
4. Forgetting default zero values

15. Builds To (Related LeetCode Problems)

Progression:

- LC 739 → Daily Temperatures
- LC 496 → Next Greater Element I
- LC 503 → Next Greater Element II
- LC 901 → Online Stock Span
- LC 84 → Largest Rectangle in Histogram

16. Alternate Approaches + Comparison

Approach	Time	Space	Notes
Brute Force	$O(n^2)$	$O(1)$	Too slow
Monotonic Stack	$O(n)$	$O(n)$	Optimal

17. Why This Solution Works (Short Intuition)

The stack maintains unresolved temperatures in decreasing order. Whenever a warmer temperature appears, all smaller temperatures can immediately determine their waiting days.

18. Variations / Follow-Up Questions

1. Find previous warmer day
2. Find next smaller temperature
3. Return actual temperatures instead of days



4. Circular temperature array
5. Dynamic temperature updates
- 6.